Streamlit

Build Beautiful Web Apps for Data Science & ML

Data Science | Machine Learning | Pythonic | Fast & Easy | No HTML/CSS Required

What is Streamlit?

Streamlit is an **open-source** app framework for Machine Learning and Data Science projects. It allows you to create **beautiful** web applications for your ML and data science projects using **Python** scripts with minimal effort.

Key Features

- Simple & intuitive – Write apps in pure Python
- No front-end experience required
- Rich set of interactive widgets
- Real-time updates & auto-reload
- Seamless data visualization
- Deploy easily (Streamlit Community Cloud, Docker, etc.)

How It Works

Write Python Script — Build your app using Streamlit APIs

→ **Run with Streamlit** — streamlit run app.py

→ **Web App Launched** — Access your app in the browser

→ **Auto-Update** — Changes in code reflect instantly

Streamlit takes care of the web framework, so you can **focus on your data, models, and creativity.**

Getting Started

1 Install Streamlit

```
pip install streamlit
```

2 Import Library

```
import streamlit as st
```

3 Create Python File

```
app.py
```

4 Run Your App

```
streamlit run app.py
```

5 Open in Browser

http://localhost:8501

By default, Streamlit apps run on **http://localhost:8501**

Important Streamlit Components

Component	Code Example	Description	Icon
Title	`st.title("Hello Streamlit")`	Displays a title (H1)	T
Write	`st.write("This is a text.")`	Displays text, markdown, DataFrame, etc.	≡
DataFrame	`st.write(df)`	Displays a dataframe as an interactive table	▦
Line Chart	`st.line_chart(data)`	Displays a line chart	↗
Sidebar	`st.sidebar.title("Menu")`	Adds components to the sidebar	▤
Button	`st.button("Click Me")`	Creates a clickable button	☝
Selectbox	`st.selectbox("Choose", options)`	Dropdown selection	▾

Example: Simple Streamlit App

app.py

```
import streamlit as st
import pandas as pd
import numpy as np

st.title("Hello Streamlit")
st.write("Hey! This is a simple text.")

df = pd.DataFrame({
    'First Column': [1,2,3,4],
    'Second Column': ['A','B','C','D']
})
st.write("Here is the DataFrame:")
st.write(df)

chart_data = pd.DataFrame(
    np.random.randn(20, 3),
    columns=['A', 'B', 'C']
)

st.line_chart(chart_data)
```

Prepared by: Dr. Michael Mahesh K.

Streamlit ML App

Build Interactive Machine Learning Web Apps with Python

No HTML/CSS Required | Rapid Development | Interactive UI Components | Perfect for ML Projects | Open Source & Free

What is Streamlit?

Streamlit is an **open-source** app framework for Machine Learning and Data Science projects. It allows you to create beautiful and interactive web applications for your ML projects using only **Python** scripts.

Key Features

- Write apps entirely in Python
- No front-end (HTML/CSS/JS) required
- Rich set of interactive widgets
- Real-time updates & auto-reload
- Seamless data visualization
- Deploy easily (Streamlit Community Cloud, Docker, etc.)

How Streamlit Works

Write Python Script — Build your app using Streamlit APIs

→ **Run with Streamlit** — `streamlit run app.py`

→ **Web App Launched** — Access your app in the browser

→ **Auto-Update** — Changes in code reflect instantly

Focus on your **data, models, and creativity** — Streamlit takes care of the web framework.

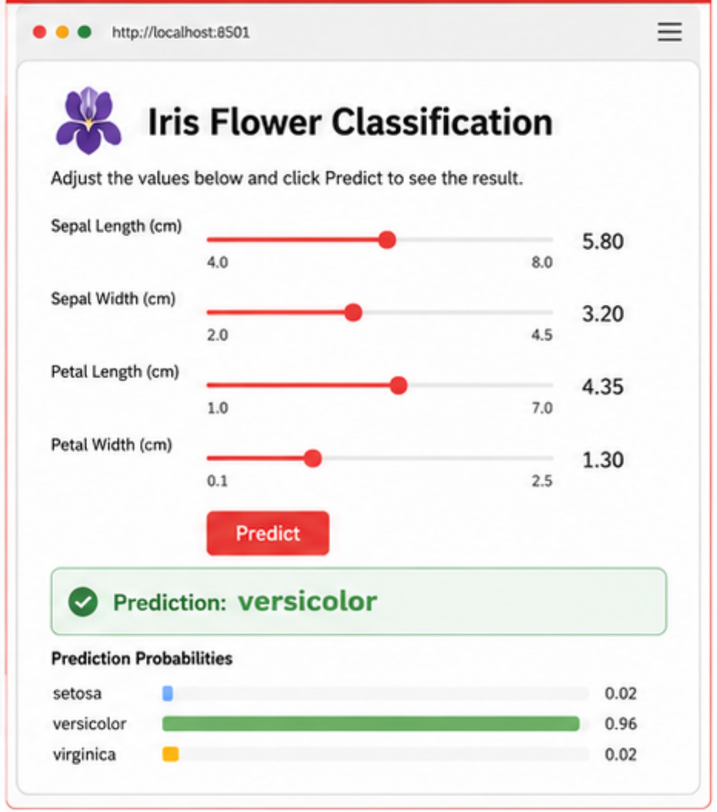
End-to-End ML App Workflow

1. **Load Dataset** (e.g., Iris Dataset)
2. **Preprocess Data** (Create DataFrame, Targets)
3. **Train Model** (Random Forest Classifier)
4. **Take User Inputs** (Sliders for Features)
5. **Predict & Display** (Result & Probabilities)

Key Code Highlights

```python
import streamlit as st
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier

# Load data
@st.cache_data
def load_data():
    iris = load_iris()
    df = pd.DataFrame(iris.data, columns=iris.feature_names)
    target = pd.Series(iris.target, name="species")
    return df, target, iris.target_names

df, target, target_names = load_data()

# Train model
model = RandomForestClassifier(random_state=42)
model.fit(df, target)

# User inputs
sepal_length = st.slider("Sepal Length", float(df["sepal length (cm)"].min()),
                         float(df["sepal length (cm)"].max()))
sepal_width = st.slider("Sepal Width", float(df["sepal width (cm)"].min()),
                        float(df["sepal width (cm)"].max()))
petal_length = st.slider("Petal Length", float(df["petal length (cm)"].min()),
                         float(df["petal length (cm)"].max()))
petal_width = st.slider("Petal Width", float(df["petal width (cm)"].min()),
                        float(df["petal width (cm)"].max()))

# Prediction
input_data = [[sepal_length, sepal_width, petal_length, petal_width]]
prediction = model.predict(input_data)[0]
prediction_proba = model.predict_proba(input_data)[0]
```

Streamlit App UI (Example)



Why Use Streamlit for ML Projects?

- Focus on what matters: Data & Models
- Build & iterate faster than traditional web development
- Great for demos, prototypes & internal tools
- Easy to share and deploy anywhere
- Backed by a strong open-source community

Popular Use Cases

- ML Model Demos
- Model Monitoring Tools
- Data Analysis Dashboards
- Chatbots & LLM Apps

Prepared by: Dr. Michael Mahesh K.

SETTING UP THE ENVIRONMENT FOR YOLO IMAGE SEARCH PROJECT

GPU (NVIDIA) INSTALLATION GUIDE

1. PREREQUISITES

- Anaconda / Miniconda installed
- Visual Studio Code installed
- NVIDIA GPU recommended for faster processing
- Project files:
  instructions.txt
  requirements.txt

2. INSTALLATION STEPS (GPU)

1. **Create conda environment**

```
conda create -n yolovimage_search_gpu python=3.11 -y
```

2. **Activate environment**

```
conda activate yolovimage_search_gpu
```

3. **Install PyTorch with CUDA** (via conda)

```
conda install pytorch torchvision pytorch-cuda=12.4
-c pytorch -c nvidia
```

Conda will automatically install the NVIDIA CUDA Toolkit version required for the selected PyTorch version.

4. **Install other dependencies**

```
pip install -r requirements.txt
```

3. CPU INSTALLATION (ALTERNATIVE)

Steps 1 & 2 are the same.

Then directly run:

```
pip install -r requirements.txt
```

requirements.txt already includes torch (CPU version), so no need to install PyTorch separately.

4. VERIFY GPU ACCESS

Python (in activated environment)

```
>>> import torch
>>> torch.cuda.is_available()
True
```

NVIDIA-SMI (in terminal)

```
+-----------------------------------------+------------------------+----------------------+
| NVIDIA-SMI 550.54.14              Driver Version: 550.54.14    CUDA Version: 12.8     |
+-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|                                         |                        |               MIG M. |
+=========================================+========================+======================+
|   0  NVIDIA GeForce RTX 4070  On        | 00000000:01:00.0 On    |                  N/A |
|  0%   45C    P8             20W / 200W  |   1234MiB / 12282MiB   |      5%      Default |
+-----------------------------------------+------------------------+----------------------+
```

Make sure your GPU driver supports the CUDA version installed. Higher driver versions are backward compatible.

5. HELPER COMMANDS

Remove Environment

```
conda remove -n <env_name> --all
```

List All Environments

```
conda env list
```

Clear Screen (Terminal)

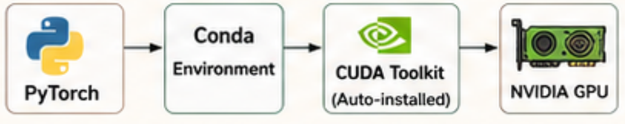
Windows : `cls` | Mac/Linux : `clear`

Check GPU (Alternate)

```
nvidia-smi
```

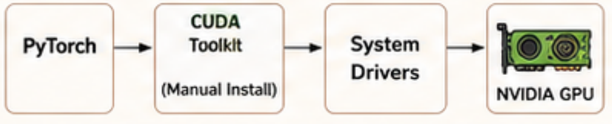
6. HOW CONDA GPU INSTALLATION WORKS

6A. With Conda (Recommended)



- Conda installs the exact CUDA Toolkit version required for the specific PyTorch version.
- No need to manually match system drivers and CUDA versions.

6B. Without Conda (Manual Installation)



- You must manually ensure compatibility between: PyTorch version, CUDA version, and NVIDIA Driver.
- May require multiple trials to find the right combination.

7. USEFUL LINKS & RESOURCES

- **PyTorch Installation (Official)**
  https://pytorch.org/get-started/locally/
- **CUDA Installation Guide (Windows)**
  https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/
- **NVIDIA Developer Forums**
  https://forums.developer.nvidia.com/
- **CUDA Toolkit & PyTorch Compatibility**
  https://forums.developer.nvidia.com/t/cuda-toolkit-and-pytorch-compatibility/

KEY TAKEAWAYS

- Use Conda for easier GPU setup. It handles CUDA Toolkit dependencies automatically.
- CPU installation is simpler and requires fewer steps, but won't use your GPU.
- Always verify `torch.cuda.is_available()` to confirm PyTorch can access your GPU.
- Read the guides and documentation for deeper understanding and troubleshooting.

Prepared by: Dr. Michael Mahesh K.

STREAMLIT BASICS FOR COMPUTER VISION PROJECTS

Build Interactive Web Apps with Pure Python – No Frontend Experience Needed!

Streamlit

1. WHY STREAMLIT FOR CV PROJECTS?

- **Quickly prototype interfaces**
  Build UIs in minutes, not days.
- **Visualize model outputs**
  Display images, charts, metrics easily.
- **Create interactive demos**
  Impress stakeholders with live demos.
- **Build tools for your team**
  Empower non-technical team members.
- **100% Pure Python**
  No HTML, CSS, or JavaScript needed.

2. BASIC WIDGETS IN STREAMLIT

Text Input	Slider	Radio Button	Select Box	Button
What's your name? John	Your age 46 (0 – 100)	Gender: Male / Female / Other	Interested in: CV & ML	Create Profile
`st.text_input()`	`st.slider()`	`st.radio()`	`st.selectbox()`	`st.button()`
Get text from user.	Select a value from a range.	Choose one option from many.	Choose one option from a dropdown.	Trigger an action on click.

3. EXAMPLE APP PREVIEW (Live UI)

Streamlit Intro

This is where we will learn Streamlit basics.

What's your name? John

Your age 46 (0 – 100)

Interested in: CV & ML

Gender: Male / Female / Other

Create Profile

Show All Profiles

Here are all the profiles:

- {'name': 'John', 'age': 46, 'gender': 'Male', 'interest': 'CV & ML'}

4. HOW STREAMLIT WORKS (RERUN MODEL)

User interacts with widget → **Streamlit**: Reruns the entire script from top to bottom → **Widgets**: Widget values are preserved automatically → **Python Variables**: Re-initialized on every rerun

Key Point: Every interaction triggers a rerun. Widgets remember their values, but normal Python variables do not.

5. SESSION STATE – SOLUTION

Session State (Persistent Dictionary)

```
if 'profiles' not in st.session_state:
    st.session_state.profiles = []
```

```
st.session_state.profiles.append(new_profile)
```

```
st.session_state.profiles
(Persists across reruns)
```

Session state stores data across reruns in a user session.

6. SAMPLE CODE SNIPPET

```python
import streamlit as st

st.title("Streamlit Intro")
st.write("This is where we will learn Streamlit basics.")

name = st.text_input("What's your name?")
age = st.slider("Your age", 0, 100, 25)
gender = st.radio("Gender", ("Male", "Female", "Other"))
interest = st.selectbox("Interested in", ("CV", "ML", "Both"))

if st.button("Create Profile"):
    new_profile = {"name": name, "age": age,
                   "gender": gender, "interest": interest}
    st.session_state.profiles.append(new_profile)

if st.button("Show All Profiles"):
    st.write("Here are all the profiles:")
    st.write(st.session_state.profiles)
```

7. KEY TAKEAWAYS

- **Rerun Behavior:** Streamlit reruns the entire script on every interaction but smartly preserves widget states.
- Entire script runs top-to-bottom on each interaction.
- Widget values persist automatically between runs.
- **Session State:** Use `st.session_state` to persist values between reruns.
- **Widget Integration:** Text inputs, sliders, radio buttons, select boxes, and buttons integrate seamlessly with your Python logic.

Interact → Rerun → Update → Repeat

8. APPLIED IN OUR PROJECT

Upload / Input → CV Model Processing → Visualize Results → User Interaction → Session State Management

These concepts power our image search interface and many other CV tools!

9. RUN YOUR STREAMLIT APP

Run from terminal:

```
streamlit run
test/streamlit_basics.py
```

Open in browser:
http://localhost:8501
(or)
http://127.0.0.1:8501

Streamlit handles the web complexity – you focus on building amazing CV solutions!

Prepared by: Dr. Michael Mahesh K.

YOLO IMAGE SEARCH PROJECT – SYSTEM ARCHITECTURE

A 5-Stage Pipeline for Scalable, Maintainable & User-Friendly Computer Vision Applications

THE 5-STAGE PIPELINE

1. INPUT STAGE — Where image enters our system.

2. PROCESSING STAGE — Where YOLO works its magic (Object Detection).

3. METADATA STAGE — Where we extract and store information.

4. QUERY STAGE — Where users search for specific content.

5. OUTPUT STAGE — Where results are presented.

ARCHITECTURAL GOALS

We split these stages into smaller components to achieve:

- Modularity
- Scalability
- Maintainability
- High Performance
- Great Usability

HIGH-LEVEL PROJECT FLOW

USER INPUT — User starts the application

PATH 1: PROCESS NEW IMAGES

1. **Select Option** — Process New Images
2. **Provide Inputs** — • Image Folder Path • Model Version (YOLOv7-M)
3. **YOLOv7 Inference (Processing Stage)** — Run detection on all images
   - Processing... (Loading Screen)
4. **Parse & Save Metadata (Metadata Stage)** — Extract classes & counts Save to JSON file
   - Processed 500 images Metadata saved to: ./metadata/yolov7_m_results.json
5. **Search & Filter (Query Stage)** — Apply filters (Class, Logic, Thresholds)
6. **Display Results (Output Stage)** — Render results grid + Export Option

PATH 2: LOAD EXISTING METADATA

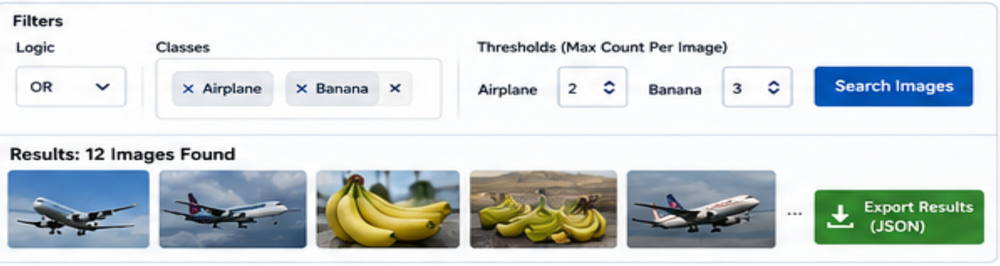
1. **Select Option** — Load Existing Metadata
2. **Provide Input** — Provide Metadata (JSON File Path)
3. **Load & Parse Metadata (Metadata Stage)** — Read JSON file Extract classes & counts
   - Successfully loaded metadata for 500 images
4. **Search & Filter (Query Stage)** — Apply filters (Class, Logic, Thresholds)
5. **Display Results (Output Stage)** — Render results grid + Export Option

Key Idea: Run inference once (Path 1), then query infinitely (Path 2).

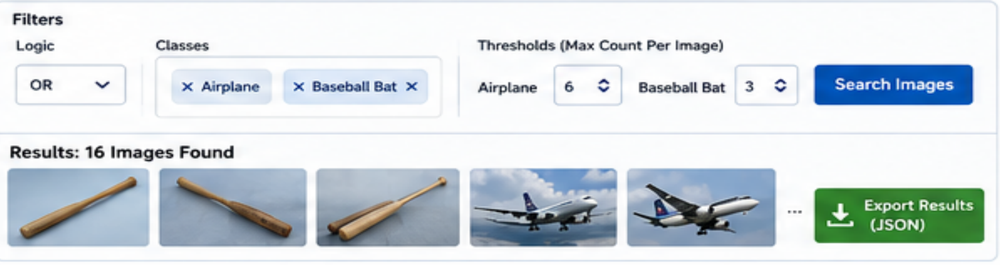
- Saves massive computation time.
- Share metadata JSON with teammates.
- Team members with CPU can still search and analyze.
- Scalable, efficient & collaborative workflow.

FINAL APPLICATION – WHAT USERS WILL SEE

A. PROCESS NEW IMAGES – EXAMPLE



B. LOAD EXISTING METADATA – EXAMPLE



KEY TAKEAWAYS

Modularity — Each stage is independent and well-defined.

Scalability — Process once, query many times.

Maintainability — Clear separation of concerns makes the project easier to maintain.

Performance — Avoid redundant inference; reuse metadata.

Usability — Intuitive UI with filters, results grid, and export capability.

Prepared by: Dr. Michael Mahesh K.

YOLO-POWERED IMAGE SEARCH APP – UI FLOW & SESSION STATE ARCHITECTURE

Designing the User Experience Before Writing Code

YOLO

UI FLOW WALKTHROUGH (PHASES)

PHASE 1: APPLICATION SETUP

1 App Launch

Initialize Session State

2 Main Options (Radio)

- Process New Images
- Load Existing Metadata

3A Process New Images

- Image Directory Path
- Model Weights Path
- Start Inference (Button)

3B Load Existing Metadata

- Metadata File Path
- metadata.json
- Load Metadata (Button)

4 Show Spinner / Loading

Running object detection...

OR

Show Spinner / Loading

Loading metadata...

5 Extract from Metadata (JSON)

- Unique Classes
- Count Options per Class (per image)

6 Save to Session State

`metadata` `unique_classes` `count_options`

Both paths output the same data → ready for search interface.

PHASE 2: SEARCH INTERFACE

1 Search Mode (Radio)

- (•) OR (Any of selected classes)
- ( ) AND (All selected classes)

Session State Key: `search_params.mode`

2 Select Classes (Multi-Select)

× Airplane × Bed

Session State Key: `search_params.classes`

3 Thresholds (Max Count Per Image)

Airplane (max): 6

Bed (max): 1

Session State Key: `search_params.thresholds`

4 Search Images (Button)

Search Images

Session State Key: `search_results.paths`

What Happens on Search?

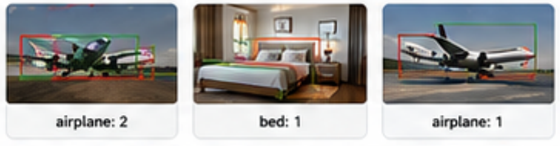
1. Filter images based on selected classes, mode & thresholds
2. Collect matching image paths
3. Save results to session state
4. Navigate to results display (Phase 3)

PHASE 3: RESULTS DISPLAY

1 Display Controls

- ☑ Show Bounding Boxes — SS Key `ui.show_boxes`
- Grid Columns: 3 — SS Key `ui.grid_cols`
- ☑ Highlight Matching Classes — SS Key `ui.highlight`

2 Render Image Grid



airplane: 2 | bed: 1 | airplane: 1

apple: 3, person: 1 | dining table: 1, person: 2 | airplane: 6

- Draw annotations (if enabled)
- Display card with metadata (class counts, total objects, etc.)
- Images converted to base64 for UI rendering

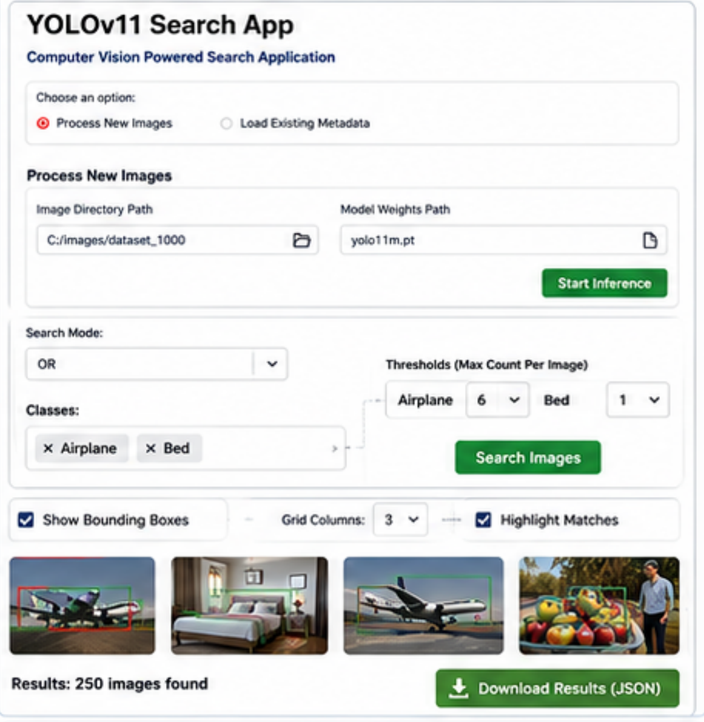
3 Export Results (Optional)

Download Results (JSON)

SS Key: `search_results.paths`

Users can review, analyze and export filtered results.

OUR YOLO IMAGE SEARCH APPLICATION (FINAL UI)



SESSION STATE OVERVIEW

Key	Purpose
metadata	All image metadata (list of dicts)
unique_classes	List of all unique classes found
count_options	Dict: class → list of unique count values
search_params.mode	OR or AND
search_params.classes	Selected classes
search_params.thresholds	Max count per class
search_results.paths	Filtered image paths
ui.show_boxes / ui.grid_cols / ui.highlight	UI preferences for results display

KEY INSIGHT

We design the user experience first, then the code. Mapping the UI flow and session state early ensures our app is intuitive, responsive, and maintainable.

- **User-Centric Design** — Plan the workflow from the user's perspective.
- **Session State** — Persist important data across reruns.
- **Clear Data Flow** — Input → Process → Filter → Render → Export
- **Modularity** — Each component handles one responsibility.
- **Better Development** — Reduces confusion and speeds up implementation.

Prepared by: Dr. Michael Mahesh K.

YOLO Image Search Application

Search UI Flow & Dynamic Filtering

Building Interactive Search & Filtering using Streamlit Session State

1. REVISITING UI FLOW ARCHITECTURE

PHASE 1 Application Setup
- Load Metadata JSON
- Extract Unique Classes
- Extract Count Options
- Save Metadata to Session State

PHASE 2 Search Interface
- Search Mode (OR / AND)
- Multi-Select Classes
- Threshold Filters
- Search Button
- Generate Filtered Results

PHASE 3 Result Display
- Bounding Box Rendering
- Grid Controls
- Highlight Matches
- Export Results JSON

Streamlit Session State (Persistent Across Reruns)

2. METADATA-BASED SEARCH SYSTEM

Metadata JSON (From Inference or File Load) → Unique Classes (Ex: airplane, apple, banana...) → Count Options (Per Class in Each Image) (Ex: 1, 2, 3, 6...) → Search Filters (User Input) → Matching Images (Filtered Results)

How It Works
- Load metadata JSON
- Extract unique classes and count options
- Populate dynamic UI widgets for filtering
- Filter images based on user-selected criteria

3. DYNAMIC SEARCH UI COMPONENTS (STREAMLIT)

Search Mode
- (•) OR (Any selected classes)
- ( ) AND (All selected classes)

Select Classes (Multi-Select)
× Airplane × Apple × Banana

Thresholds (Max Count Per Image)

Airplane (max)	Apple (max)	Banana (max)
1	2	3

Search Images

Widgets maintain their own state while selected values are stored in Streamlit Session State.

4. SESSION STATE ARCHITECTURE

Why Session State?
- Prevent data loss during reruns (Streamlit re-executes the script on every interaction)
- Share data between different UI components
- Maintain user selections, filters and results
- Provide persistent app behavior

```
search_params = {
   "search_mode":  "OR",
   "select_classes": [],
   "thresholds": {}
}
        # Dictionary in Session State
```

```
search_results = []
    # List of matching image metadata
```

All important data stored in session_state and survives across interactions!

5. THRESHOLD FILTERING MECHANISM

Selected Class → Extract Detections (from image) → Count Matching Objects → Compare with Threshold → TRUE / FALSE Match

Example 1
Threshold = 4
Class Count = 2
Result = TRUE

Example 2
Threshold = 6
Class Count = 10
Result = FALSE

A match is valid when:
1 ≤ Class Count ≤ Threshold (If threshold is set)
If no threshold selected (None) → any count ≥ 1 is valid

6. OR vs AND SEARCH LOGIC

OR MODE (Any Match)
Uses Python any()

At least one selected class must match

Example:
Apple = FALSE
Banana = TRUE
Final Result = TRUE

More flexible – returns images matching any selected class.

VS

AND MODE (All Match)
Uses Python all()

All selected classes must match together

Example:
Apple = TRUE
Banana = TRUE
Final Result = TRUE

More strict – returns images matching all selected classes.

7. FILTERED RESULT GENERATION PIPELINE

Metadata (List of Images) → Iterate Images → Filter Classes → Apply Thresholds → Apply OR/AND Logic → Append Matching Results → Save to search_results (Session State)

This pipeline runs when user clicks "Search Images" button.

8. SEARCH RESULT PREPARATION

Image Card Example

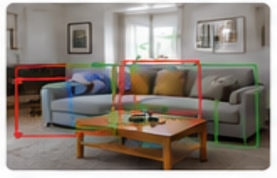


image_0123.jpg
3 objects
- Apple : 1
- Person : 2
- Bottle : 1
...

What's Included?
- Image Preview (Base64 Encoded)
- Detected Classes
- Object Counts
- Total Objects
- All Detections
- Bounding Boxes (Optional)

Filtered results are passed to the final UI rendering module for grid display, controls and export.

9. KEY TECHNICAL CONCEPTS

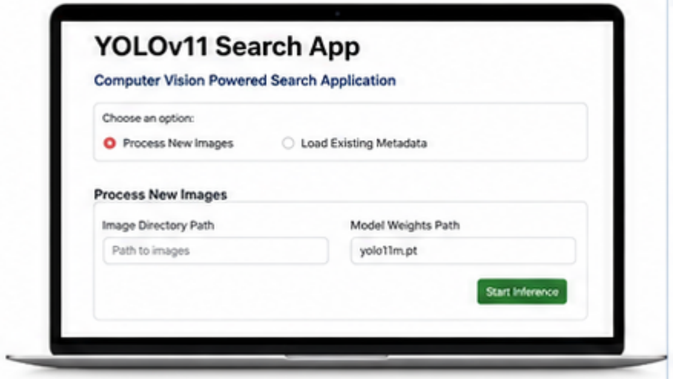
- **Streamlit Session State** – Persist data across reruns
- **Dynamic Widget Population** – Populate UI based on metadata
- **Metadata Filtering** – Filter images using classes & counts
- **Boolean Search Logic** – OR (any) vs AND (all) logic
- **Threshold-Based Search** – Limit max count per class
- **YOLO Detection Filtering** – Use model output for smart search
- **Multi-Class Query System** – Select multiple classes at once
- **Interactive UI Design** – Intuitive & responsive UI

YOLO-Powered Image Search App – Backend Implementation

Lecture Summary: Building the Core Foundation

1. WHAT WE BUILT

- YOLOv11 inference backend integrated with Streamlit
- Process new images OR load existing metadata
- Extract detections, counts, unique classes & options
- Save metadata to JSON
- Load metadata and reuse without re-running inference
- Ready for search, filtering & visualization



Images → YOLO Inference → Metadata (JSON) → Search & Filter → Results Grid

2. SYSTEM ARCHITECTURE

Five-Stage Pipeline

1. **INPUT STAGE** – Where image enters our system
2. **PROCESSING STAGE** – Where YOLO works its magic
3. **METADATA STAGE** – Where we extract & store information
4. **QUERY STAGE** – Where users search for specific content
5. **OUTPUT STAGE** – Where results are presented

Data Flow

User Input → YOLO Inference → Parse & Store Metadata (JSON) → Apply Filters / Query → Render Results / Visualization

3. HIGH LEVEL PROJECT FLOW

PATH 1: Process New Images

1. User selects Process New Images (Provide images path + model)
2. Run YOLO Inference (on all images)
3. Parse Detections & Extract Classes/Counts
4. Save Metadata (JSON) to local directory
5. Apply Filters / Search (Classes, Thresholds)
6. Render Results Grid & Export JSON (Optional)

PATH 2: Load Existing Metadata

1. User selects Load Existing Metadata (Provide JSON path)
2. Load Metadata (JSON) from disk
3. Extract Classes & Counts from Metadata
4. Apply Filters / Search (Classes, Thresholds)
5. Render Results Grid & Export JSON (Optional)

4. KEY CODE COMPONENTS

app.py (Streamlit UI)

- Page config & title
- Radio: Process New / Load Existing
- Text inputs for paths
- Start Inference / Load Metadata
- Session state management
- Success messages & spinners

```
import streamlit as st
st.set_page_config(page_title="YOLOv11 Search App", layout="wide")

option = st.radio("Choose an option:", ["Process New Images", "Load Existing Metadata"], horizontal=True)
```

inference.py (YOLO Backend)

- Load YOLO model (CUDA by default)
- Process directory → iterate images
- Run inference on each image
- Extract detections, counts, classes
- Return metadata list

```
results = self.model.predict(
    source=image_path,
    conf=self.conf_threshold,
    device=self.device
)
```

utils.py (Utilities)

- ensure_processed_directory()
- save_metadata(metadata, path)
- load_metadata(path)
- get_unique_classes_and_counts(metadata)
- Robust file & path handling

```
with open(output_path, 'w') as f:
    json.dump(metadata, f)

return output_path
```

5. METADATA STRUCTURE (JSON)

```
{
  "image_path": "C:/images/0001.jpg",
  "detections": [
    {"class": 0, "confidence": 0.91, "bbox": [120.5, 80.2, 210.7, 160.3], "count": 2},
    {"class": 2, "confidence": 0.87, "bbox": [300.1, 150.6, 420.5, 290.2], "count": 1}
  ],
  "total_objects": 3,
  "unique_classes": [0, 2],
  "class_counts": {
    "0": 2,
    "2": 1
  }
}
```

Contains everything needed for:

- Rendering images with bounding boxes
- Filtering by class & count thresholds
- Building unique class & count options

Example Meaning:

- Class index 0 appears 2 times in this image
- Class index 2 appears 1 time in this image

6. WHAT'S NEXT?

1. **Build Search Engine UI** – Class selection, OR/AND logic, threshold filters
2. **Filter & Query Logic** – Apply filters on metadata to get matching images
3. **Render Results Grid** – Show images with bounding boxes & counts
4. **Export JSON (Optional)** – Allow users to download filtered results
5. **Enhance UI/UX** – Pagination, performance, visual polish

Key Takeaway

We built a robust backend that processes images, extracts meaningful metadata, and prepares everything for a powerful image search experience.

CONCEPTS COVERED
Streamlit • YOLO Inference • Metadata • JSON • Session State • File Handling

BEST PRACTICES
Modular Code • Error Handling • Config Management • Reusability

RESULT
Backend ready for scalable, maintainable & user-friendly image search.

Prepared by: Dr. Michael Mahesh K.

WHY THIS PROJECT MATTERS & HOW IT WORKS

Understanding Use Cases, Users & Industry Workflow

YOLO

1. PROJECT USE CASES

1 Predict & Validate Model's Output

Input Image → YOLO Model (Inference) → Output (Class, Conf., Coordinates) → Visualize on Image (Validation by Human)

Helps verify if the model is detecting objects correctly.

2 Export Desired Data Based on Custom Selection

Large Image Collection → YOLO Inference (80 Classes) → Filter by Custom Classes (e.g., Cat, Dog) → Export Filtered Results (JSON)

2. WHO WILL USE THIS PROJECT?

COMMON USERS (Standard Functionality)

- **Professionals (Photographers, Creators)**
  Organize and find images quickly.
- **E-commerce Businesses**
  Manage and categorize product images efficiently.
- **Anyone Finding Specific Photos**
  Search for photos with desired objects (e.g., dog with ball).

POWER USERS (Advanced Capabilities)

- **Data Annotators**
  Use model predictions for partial annotation & review.
- **CV / DL Engineers**
  Extract custom data, analyze dataset & model quality.
- **Team Leads / Managers**
  Review annotations and ensure data quality at scale.

3. DATA ANNOTATION + VERSIONING WORKFLOW

1. **Data Collection** — Collect raw images from various sources.
2. **Data Cleaning** — Remove duplicates, irrelevant or low-quality data.
3. **Data Annotation** — Annotators label objects using tools (CVAT, Roboflow, etc.).
4. **Data Review** — Engineers review annotations for quality.
5. **Version Dataset** — Create versioned dataset when satisfied. (v1)

If more data is needed, collect more and repeat the cycle.

4. WHERE THIS PROJECT IS USED IN INDUSTRY (Example)

A. Baseline Approach (Traditional)

Goal: Model v1 with 10 classes

5,000 Clean Images (Ready for Annotation) → Full Annotation by Data Team (Time: ~ 1 Month) → Model Training (Hours / Days) → Model v1 (10 Classes) v1

B. Using Our Project (Smarter & Faster)

Goal: Model v2 with 10 classes

Add 5,000 New Clean Images (Total: 10,000 Images) → Our Project (YOLO Inference) Auto-annotates ~50% of objects → Partial Annotation / Review by Data Team (Time: ~10–15 Days) → Model Training (on 10,000 Images) (Hours / Days) → Model v2 (10 Classes) v2

Our project reduces annotation time by ~50% by auto-generating predictions, helping teams build better models faster!

5. APP UI FLOW (3 PHASES)

PHASE 1: APPLICATION SETUP

1. Choose Option (Radio)
   - Process New Images
   - Load Existing Metadata

Session State Key: `app_choice`

Process New Images
- Image Directory Path
- Model Weights Path (.pt/.onnx)
- Start Inference (Button)

B. Load Existing Metadata
- Metadata File Path
- Load Metadata (Button)

Session State Key: `metadata_path`

Show Spinner / Loading

Extract from Metadata (JSON)
- Unique Classes
- Count Options per Class

Session State Keys: `unique_classes`, `count_options`

PHASE 2: SEARCH INTERFACE

1. Search Mode (Radio) — OR ○ AND — SS Key: `search_params.mode`
2. Select Classes (Multi-Select) — SS Key: `search_params.classses`
3. Thresholds (Max Count Per Image) — Airplane: 6 Bed: 1 — SS Key: `search_params.thresholds`
4. Search Images (Button) — SS Key: `search_results.paths`

PHASE 3: RESULTS DISPLAY

1. Display Controls
   - ☑ Show Bounding Boxes
   - Grid Columns: 3
   - ☑ Highlight Matches

   SS Keys: `ui.show_boxes`, `ui.grid_cols`, `ui.highlight`
2. Render Image Grid
   From `search_results.paths`
3. Export Results (Optional)
   Download Results (JSON)

   SS Key: `export_path`

KEY INSIGHTS

- **Design First** — Map the UI flow and session state before writing code for a seamless user experience.
- **Session State is Key** — Preserves important data across reruns and connects all components.
- **Data Flows Smoothly** — Input → Process → Metadata → Query → Results → Export
- **Saves Time & Effort** — Auto-annotation & smart filtering reduce manual work and speed up model development.
- **Useful for Everyone** — From daily users searching photos to teams building CV products at scale.

Prepared by: Dr. Michael Mahesh K.